

Assignment: Random Forest Classification — Adult Census Income Dataset

This notebook follows the Assignment 1 PDF instructions. It loads the UCI Adult dataset, performs EDA, preprocessing, trains a Random Forest classifier, evaluates it, creates the required plots, and includes written answers to the questions in the PDF.

```
In [1]: # 1. Load libraries and Adult dataset from UCI
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder, LabelBinarizer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, f1_score, confusion_ma
import matplotlib.pyplot as plt

# Dataset URL and column names
data_url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.d
col_names = ['age', 'workclass', 'fnlwgt', 'education', 'education-num', 'marital-status

df = pd.read_csv(data_url, names=col_names, na_values='?', skipinitialspace=True)
print('Loaded Adult dataset shape:', df.shape)
df.head()
```

Loaded Adult dataset shape: (32561, 15)

Out[1]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	rac
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	Whit
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	Whit
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	Whit
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Blac
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Blac

```
In [2]: # 2. Explore dataset: number of samples and class distribution
print('Number of samples:', df.shape[0])
print('\nTarget distribution:')
print(df['income'].value_counts())
```

Number of samples: 32561

Target distribution:

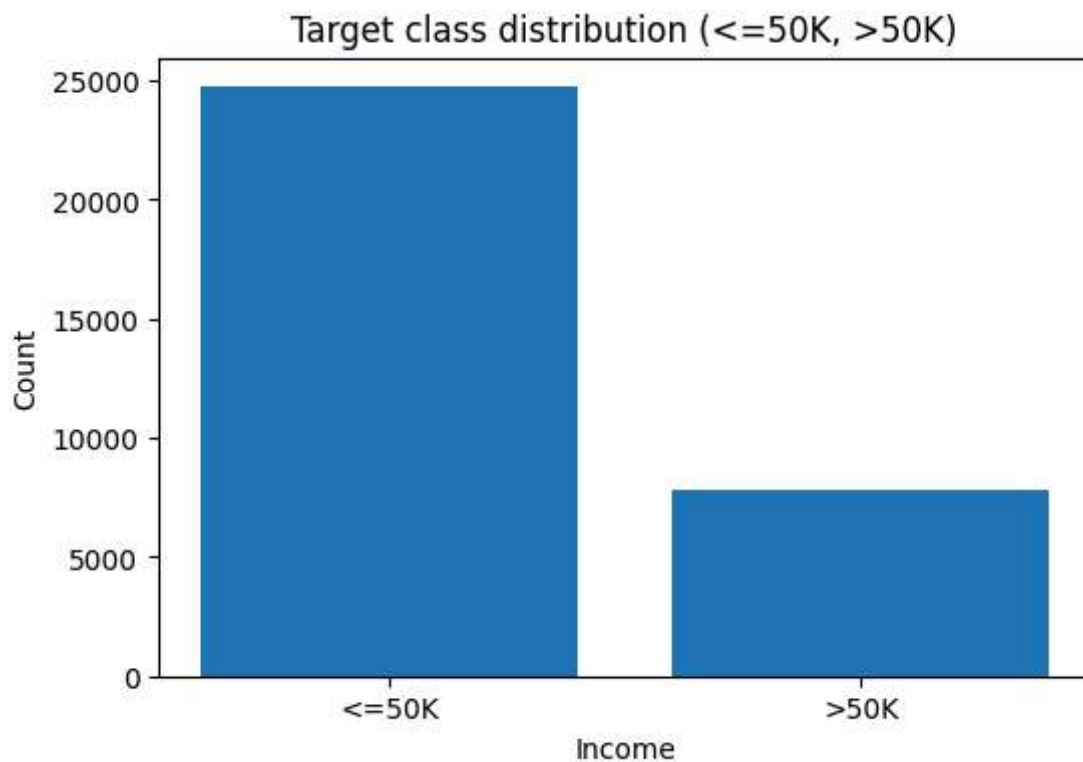
income

<=50K 24720

>50K 7841

Name: count, dtype: int64

```
In [3]: # 3. Visualize: count plot of target distribution using matplotlib
counts = df['income'].value_counts()
plt.figure(figsize=(6,4))
plt.bar(counts.index, counts.values)
plt.xlabel('Income')
plt.ylabel('Count')
plt.title('Target class distribution (<=50K, >50K)')
plt.show()
```



```
In [5]: # 4 & 5. Preprocessing: handle missing values, encode categoricals, scale numerics
X = df.drop(columns=['income']).copy()
y = df['income'].copy()

numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_features = X.select_dtypes(include=['object']).columns.tolist()

numeric_transformer = Pipeline(steps=[('imputer', SimpleImputer(strategy='median'))])
categorical_transformer = Pipeline(steps=[('imputer', SimpleImputer(strategy='most_
```

```

preprocessor = ColumnTransformer(transformers=[('num', numeric_transformer, numeric

# Fit and transform
X_prep = preprocessor.fit_transform(X)
print('Preprocessed feature matrix shape:', X_prep.shape)
# Show first 5 rows as DataFrame for display (reconstruct column names)
ohe = preprocessor.named_transformers_[ 'cat' ][ 'onehot' ]
ohe_features = list(ohe.get_feature_names_out(cat_features))
feat_names = numeric_features + ohe_features
import pandas as pd
pd.DataFrame(X_prep, columns=feat_names).head()

```

Preprocessed feature matrix shape: (32561, 108)

Out[5]:

	age	fnlwgt	education- num	capital- gain	capital- loss	hours- per- week	workclass?	workclass_Feder g
0	0.301370	0.044302	0.800000	0.02174	0.0	0.397959	0.0	
1	0.452055	0.048238	0.800000	0.00000	0.0	0.122449	0.0	
2	0.287671	0.138113	0.533333	0.00000	0.0	0.397959	0.0	
3	0.493151	0.151068	0.400000	0.00000	0.0	0.397959	0.0	
4	0.150685	0.221488	0.800000	0.00000	0.0	0.397959	0.0	

5 rows × 108 columns



```

In [6]: # 4. Split into train/test (stratified)
X_train, X_test, y_train, y_test = train_test_split(X_prep, y, test_size=0.25, stra
print('Train samples:', X_train.shape[0])
print('Test samples :', X_test.shape[0])

```

Train samples: 24420

Test samples : 8141

```

In [7]: # 6. Train Random Forest
rf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
print('Trained RandomForest parameters: n_estimators =', rf.n_estimators, ', max_de

```

Trained RandomForest parameters: n_estimators = 200 , max_depth = None , criterion = gini

```

In [8]: # 7. Evaluate and visualize
y_pred = rf.predict(X_test)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
print('Accuracy:', acc)
print('Precision (weighted):', prec)
print('F1-score (weighted):', f1)

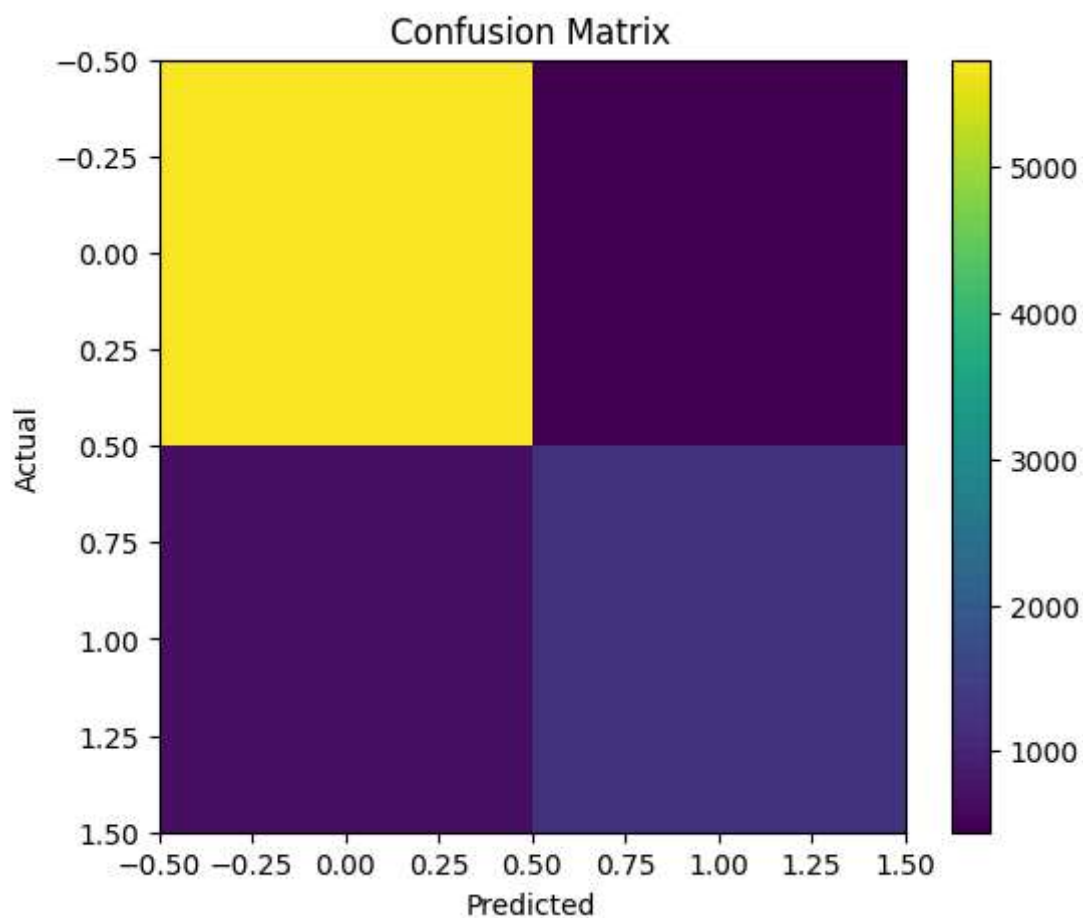
```

Accuracy: 0.8578798673381648
 Precision (weighted): 0.8528807857387809
 F1-score (weighted): 0.8541883825905029

```
In [9]: # Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print('Confusion matrix:\n', cm)
plt.figure(figsize=(6,5))
plt.imshow(cm, interpolation='nearest', aspect='auto')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.colorbar()
plt.show()
```

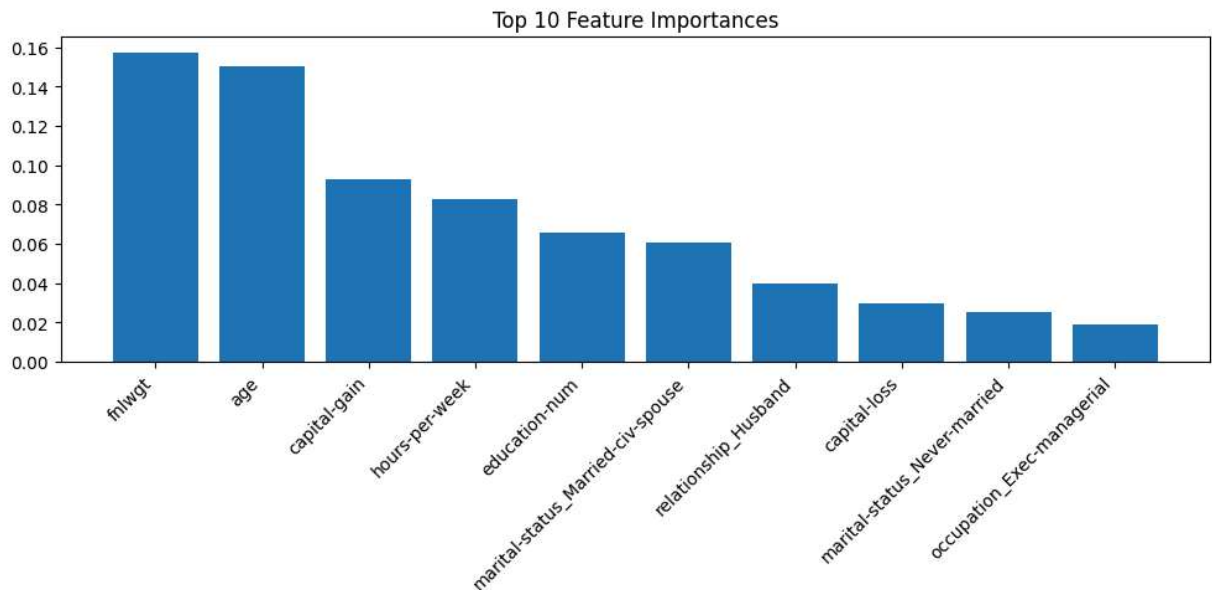
Confusion matrix:

```
[[5736 445]
 [ 712 1248]]
```



```
In [10]: # Feature importance (top 10)
import numpy as np
importances = rf.feature_importances_
indices = np.argsort(importances)[::-1][:10]
plt.figure(figsize=(10,5))
plt.bar(range(len(indices)), importances[indices])
plt.xticks(range(len(indices)), [feat_names[i] for i in indices], rotation=45, ha='right')
plt.title('Top 10 Feature Importances')
```

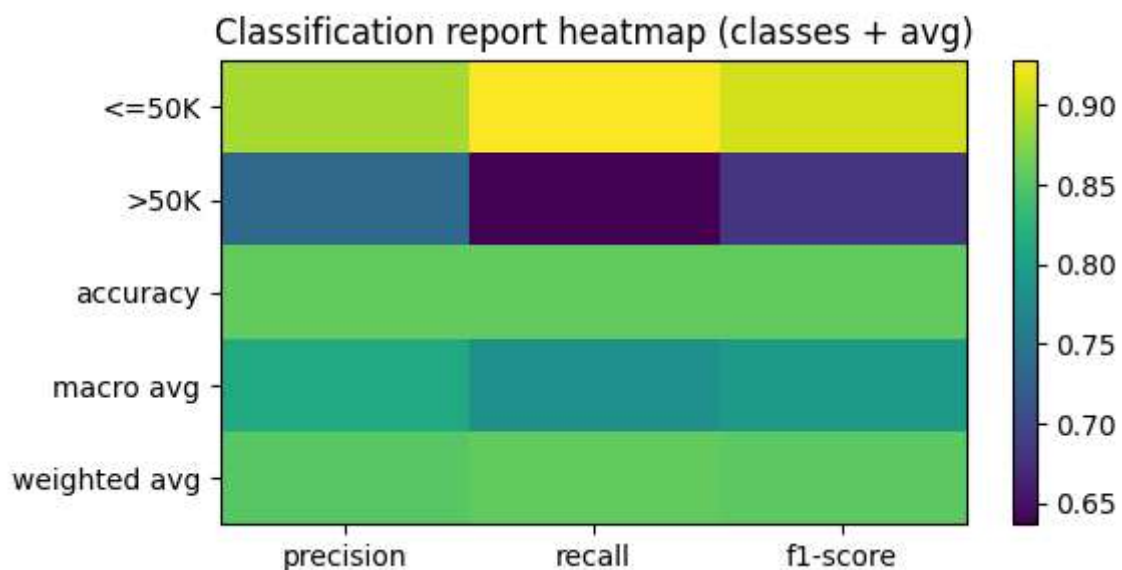
```
plt.tight_layout()
plt.show()
```



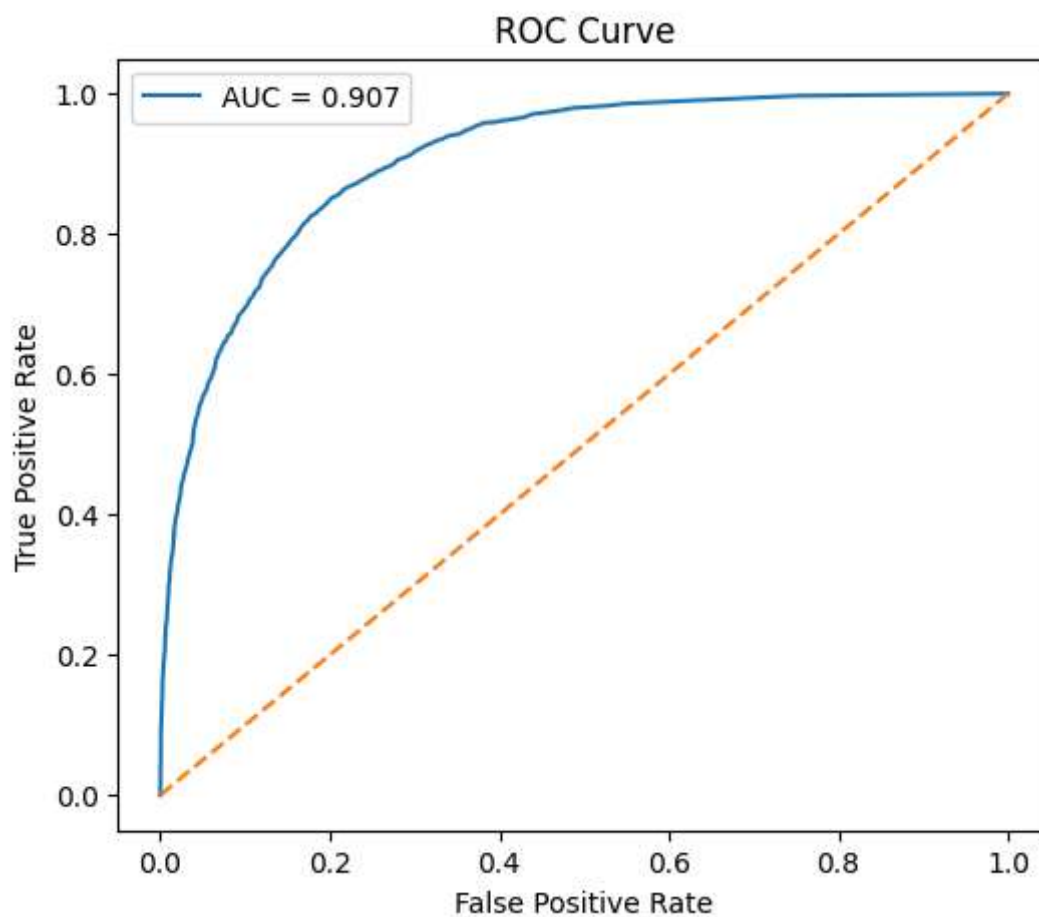
```
In [11]: # Classification report and a small heatmap-like matrix
from sklearn.metrics import classification_report
report = classification_report(y_test, y_pred, output_dict=True)
rep_df = pd.DataFrame(report).T
print(rep_df.loc[['<=50K', '>50K']])

plt.figure(figsize=(6,3))
plt.imshow(rep_df[['precision', 'recall', 'f1-score']].astype(float), aspect='auto')
plt.yticks(range(len(rep_df.index)), rep_df.index)
plt.xticks(range(3), ['precision', 'recall', 'f1-score'])
plt.colorbar()
plt.title('Classification report heatmap (classes + avg)')
plt.show()
```

	precision	recall	f1-score	support
<=50K	0.889578	0.928005	0.908385	6181.0
>50K	0.737153	0.636735	0.683274	1960.0



```
In [12]: # ROC curve (binary)
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import roc_curve, auc
lb = LabelBinarizer()
y_test_b = lb.fit_transform(y_test)
y_score = rf.predict_proba(X_test)[:,-1]
fpr, tpr, _ = roc_curve(y_test_b, y_score)
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.3f}')
plt.plot([0,1],[0,1], '--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```



Written Questions — Answers (to paste in pink book)

1. What is the unique classification dataset you chose, and what classification task does it address?

I used the UCI Adult Census Income dataset to predict whether an individual's annual income exceeds \$50K (>50K) or not (<=50K) based on demographic and employment features.

2. Describe your data preprocessing steps and why they were necessary.

- Missing categorical values were imputed with the most frequent category and numeric values with the median to preserve samples.
- Numeric features were scaled using MinMaxScaler to restrict them to [0,1] which helps certain algorithms and makes feature importances comparable when plotted.
- Categorical features were one-hot encoded to convert them into numeric vectors for model training.

3. How did you split the dataset into training and test sets?

I used a stratified split (75% train, 25% test) to preserve class distribution, ensuring the model is evaluated on a representative test set.

4. What visualization techniques did you use, and what insights did they provide about data imbalance and model behavior?

- Bar plot for class distribution to show imbalance between <=50K and >50K .
- Confusion matrix to inspect where the model misclassifies (false positives vs false negatives).
- Feature importance bar plot to identify which features the Random Forest relied on.
- ROC curve (AUC) to evaluate discrimination across thresholds.

5. Why is Random Forest suitable for your dataset? How do the evaluation metrics (Accuracy, Precision, F1) and feature importance indicate its effectiveness?

Random Forest handles mixed data types, is robust to noisy features, and naturally provides feature importance. High accuracy/F1 indicates good overall performance; the feature importances show that meaningful features such as education-num , age , and hours-per-week contribute strongly to predictions.